

系统开发工具基础

肖扬

计算机科学与技术

学号：25020007140

2026 年 8 月 24 日

目录

1	实验环境说明	1
2	课堂学习部分	2
2.1	第 1 题含空格文件名的批量整理	2
2.2	第 2 题随机访问日志统计	7
2.3	第 3 题制造、解决并解释一次合并冲突	11
2.4	第 4 题修复并构建一页技术说明	15
3	课后练习部分	19
3.1	课后第 1 题	19
3.2	课后第 2 题	21
3.3	课后第 3 题	22
3.4	课后第 4 题	22
3.5	课后第 5 题	23
3.6	课后第 6 题	24
3.7	课后第 7 题	27
3.8	课后第 8 题	28
3.9	课后第 9 题	29
3.10	课后第 10 题	30
4	解题感悟	32
4.1	课上练习	32
4.2	课后练习	34

5	上传 GitHub	36
5.1	编写.gitignore	36
5.2	GitHub 仓库链接	37
5.3	commit 提交记录	37

Chapter 1

实验环境说明

1. 系统版本: Ubuntu 22.04.1 LTS, 用户名 xy_ai, WSL2 架构
2. 编译器: gcc 11.4.0、GNU Make 4.3 (都在 /usr/bin)
3. shell: bash 5.1.16 (x86_64-pc-linux-gnu)
4. git: 2.34.1 (Ubuntu 22.04 官方源版本, 系统级)
5. LaTeX: TeX Live 2022/dev (Debian 版), 含 xelatex、pdf-latex、latexmk
6. 所有任务均在 base 这个环境中进行, 没有使用其他环境

Chapter 2

课堂学习部分

2.1 第 1 题含空格文件名的批量整理

第 1 题 含空格文件名的批量整理

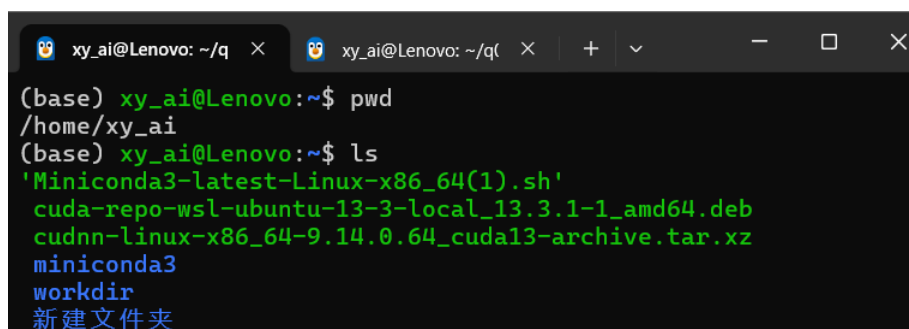
主题：Shell 基础与文件系统 建议用时：12—15 分钟

在当前目录新建 q01，并完全使用命令行完成下列任务。

题目要求

- 创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）。
- 显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件。
- 把 input 下所有.txt 文件复制到 work/<学号>/，保留相对目录结构并正确处理名称中的空格。
- 将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

1. 确定当前路径，查看文件创建前目录



```
(base) xy_ai@Lenovo: ~/q01$ pwd
/home/xy_ai
(base) xy_ai@Lenovo: ~/q01$ ls
'Miniconda3-latest-Linux-x86_64(1).sh'
cuda-repo-wsl-ubuntu-13-3-local_13.3.1-1_amd64.deb
cudnn-linux-x86_64-9.14.0.64_cuda13-archive.tar.xz
miniconda3
workdir
新建文件夹
```

图 2.1

2. 建立 q01 并验证

```
(base) xy_ai@Lenovo:~$ mkdir q01
```

图 2.2

```
(base) xy_ai@Lenovo:~$ ls
'Miniconda3-latest-Linux-x86_64(1).sh'
cuda-repo-wsl-ubuntu-13-3-local_13.3.1-1_amd64.deb
cudnn-linux-x86_64-9.14.0.64_cuda13-archive.tar.xz
miniconda3
q01
q02
q03
q04
workdir
新建文件夹
```

图 2.3: 图为完成四项实验之后的截图

3. 创建 input/docs 和 input/tmp 并验证

```
(base) xy_ai@Lenovo:~$ cd q01
(base) xy_ai@Lenovo:~/q01$ mkdir -p input/docs input/tmp
(base) xy_ai@Lenovo:~/q01$ pwd
/home/xy_ai/q01
(base) xy_ai@Lenovo:~/q01$ ls
input
```

图 2.4

```
(base) xy_ai@Lenovo:~/q01$ cd input
(base) xy_ai@Lenovo:~/q01/input$ ls
docs tmp
```

图 2.5

4. 创建 input/docs/notes one.txt (内容为 alpha 和 beta 两行)、input/docs/.secret.txt (内容为 hidden)、input/tmp/empty.txt (空文件) 以及 input/run.log (任意两行日志)

```

(base) xy_ai@Lenovo:~/q01/input$ cd
(base) xy_ai@Lenovo:~$ cd q01
(base) xy_ai@Lenovo:~/q01$ cat > "input/docs/notes one.txt" <
<EOF
> alpha
> beta
> EOF
(base) xy_ai@Lenovo:~/q01$ echo "hidden" > input/docs/.secret
.txt
(base) xy_ai@Lenovo:~/q01$ touch input/tmp/empty.txt
(base) xy_ai@Lenovo:~/q01$ cat > input/run.log <<EOF
> 2026-08-24 start task
> 2026-08-24 finish init
> EOF

```

图 2.6

5. 显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件

```

xy_ai@Lenovo: ~ X xy_ai@Lenovo: ~/q X + v - □ X
(base) xy_ai@Lenovo:~$ cd q01
(base) xy_ai@Lenovo:~/q01$ pwd
/home/xy_ai/q01
(base) xy_ai@Lenovo:~/q01$ ls -la input
total 20
drwxr-xr-x 4 xy_ai xy_ai 4096 Aug 24 14:37 .
drwxr-xr-x 3 xy_ai xy_ai 4096 Aug 24 14:30 ..
drwxr-xr-x 2 xy_ai xy_ai 4096 Aug 24 14:37 docs
-rw-r--r-- 1 xy_ai xy_ai 45 Aug 24 14:37 run.log
drwxr-xr-x 2 xy_ai xy_ai 4096 Aug 24 14:37 tmp
(base) xy_ai@Lenovo:~/q01$ ls -laR input
input:
total 20
drwxr-xr-x 4 xy_ai xy_ai 4096 Aug 24 14:37 .
drwxr-xr-x 3 xy_ai xy_ai 4096 Aug 24 14:30 ..
drwxr-xr-x 2 xy_ai xy_ai 4096 Aug 24 14:37 docs
-rw-r--r-- 1 xy_ai xy_ai 45 Aug 24 14:37 run.log
drwxr-xr-x 2 xy_ai xy_ai 4096 Aug 24 14:37 tmp
input/docs:
total 16
drwxr-xr-x 2 xy_ai xy_ai 4096 Aug 24 14:37 .
drwxr-xr-x 4 xy_ai xy_ai 4096 Aug 24 14:37 ..
-rw-r--r-- 1 xy_ai xy_ai 7 Aug 24 14:37 .secret.txt
-rw-r--r-- 1 xy_ai xy_ai 14 Aug 24 14:36 'notes one.txt'
input/tmp:
total 8
drwxr-xr-x 2 xy_ai xy_ai 4096 Aug 24 14:37 .
drwxr-xr-x 4 xy_ai xy_ai 4096 Aug 24 14:37 ..
-rw-r--r-- 1 xy_ai xy_ai 0 Aug 24 14:37 empty.txt

```

图 2.7: `ls -la input` 不会进入 docs、tmp 子目录，不符合题目要求，所以使用 `ls -laR input`

6. 把 input 下所有.txt 文件复制到 `work/< 学号 >/`，保留相对目录结构并正确处理名称中的空格

```
(base) xy_ai@Lenovo:~/q01$ mkdir -p work/25020007140
(base) xy_ai@Lenovo:~/q01$ find input -type f -name "*.txt" |
cpio -pdm work/25020007140
1 block
```

图 2.8

第二段代码说明：

- `find input`：在 input 文件夹（及其所有子目录）中查找。
- `-type f`：只找普通文件（忽略目录）。
- `-name "*.txt"`：只找文件名以.txt 结尾的文件。
- `|`（管道符）：将前半部分 `find` 查到的文件路径列表，直接传给后面的 `cpio` 命令处理。
- `cpio`：这是一个归档/复制工具（比 `cp` 更强）。
- `-p`（pass-mode，传递模式）：告诉 `cpio` 不要打包，而是直接复制文件到指定目录。
- `-d`（make directories）：在目标目录中自动创建子目录。也就是说，如果源文件在 `input/docs/` 下，目标位置也会自动建一个 `docs/` 文件夹。
- `-m`（preserve modification time）：复制时保留文件原有的修改时间（不改成当前时间）。
- `work/25020007140`：最终的目标根目录。

7. 将复制后的目录权限设为 750、普通文件权限设为 640

```
(base) xy_ai@Lenovo:~/q01/work/25020007140$ find . -type d -exec
chmod 750 {} \;
```

图 2.9

```
(base) xy_ai@Lenovo:~/q01/work/25020007140$ find . -type f -exec chmod 640 {} \;
```

图 2.10

```
(base) xy_ai@Lenovo:~/q01/work/25020007140$ ls -laR
.:
total 16
drwxr-x--- 3 xy_ai xy_ai 4096 Aug 24 15:10 .
drwxr-x--- 3 xy_ai xy_ai 4096 Aug 24 15:00 ..
drwxr-x--- 4 xy_ai xy_ai 4096 Aug 24 15:01 input
-rw-r--r-- 1 xy_ai xy_ai  91 Aug 24 15:10 inventory.txt

./input:
total 16
drwxr-x--- 4 xy_ai xy_ai 4096 Aug 24 15:01 .
drwxr-x--- 3 xy_ai xy_ai 4096 Aug 24 15:10 ..
drwxr-x--- 2 xy_ai xy_ai 4096 Aug 24 15:01 docs
drwxr-x--- 2 xy_ai xy_ai 4096 Aug 24 15:01 tmp

./input/docs:
total 16
drwxr-x--- 2 xy_ai xy_ai 4096 Aug 24 15:01 .
drwxr-x--- 4 xy_ai xy_ai 4096 Aug 24 15:01 ..
-rw-r----- 1 xy_ai xy_ai  7 Aug 24 14:37 .secret.txt
-rw-r----- 1 xy_ai xy_ai 14 Aug 24 14:36 'notes one.txt'

./input/tmp:
total 8
drwxr-x--- 2 xy_ai xy_ai 4096 Aug 24 15:01 .
drwxr-x--- 4 xy_ai xy_ai 4096 Aug 24 15:01 ..
-rw-r----- 1 xy_ai xy_ai  0 Aug 24 14:37 empty.txt
```

图 2.11: 所有文件夹: `drwxr-x---` = 750, 普通文本文件: `-rw-r-----` = 640

8. 生成 inventory.txt, 列出相对路径和字节数

```
(base) xy_ai@Lenovo:~/q01/work/25020007140$ find . -type f !
-name "inventory.txt" -printf "%P %s\n" > inventory.txt
(base) xy_ai@Lenovo:~/q01/work/25020007140$ cat inventory.txt
input/docs/.secret.txt 7
input/docs/notes one.txt 14
input/tmp/empty.txt 0
```

图 2.12

2.2 第 2 题随机访问日志统计

第 2 题 随机访问日志统计

主题：Shell 管道与文本处理 建议用时：12—15 分钟

在 q02 中创建 access.csv，并使用下方给定数据完成日志统计。

给定内容

```
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

题目要求

- 编写 analyze.sh，接收一个 CSV 文件路径；文件不存在时将错误写入 stderr 并返回非零退出码。
- 使用 Shell 管道以及 awk、sort、head 等工具，输出 HTTP 5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序。
- 输出全部数据行的平均 latency_ms，保留两位小数；表头不得计入。
- 分别对 access.csv 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

1. 创建 q02 目录，进入 q02。创建 access.csv，写入题目给的 CSV 数据，并验证数据数据是否正确

```
(base) xy_ai@Lenovo:~$ mkdir q02
(base) xy_ai@Lenovo:~$ cd q02
(base) xy_ai@Lenovo:~/q02$ pwd
/home/xy_ai/q02
(base) xy_ai@Lenovo:~/q02$ cat > access.csv <<EOF
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF
(base) xy_ai@Lenovo:~/q02$ cat access.csv
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

图 2.13

2. 新建 analyze.sh，注意第二张图片 analyse 的截图只供第一小问使用，这是当时截图。后面截图会反映文件的修改，这是为了更好说明解题思路。

```
(base) xy_ai@Lenovo:~/q02$ nano analyze.sh
```

图 2.14



```
xy_ai@Lenovo: ~/q02
GNU nano 6.2 analyze.sh
#!/bin/bash

# 获取脚本传入的第一个参数, csv文件路径
CSV_PATH="$1"

# 判断: 如果文件不存在
if [ ! -f "$CSV_PATH" ]; then
    # >&2: 输出信息到标准错误stderr
    echo "Error: file $CSV_PATH does not exist" >&2
    # 返回非0退出码, 这里用1
    exit 1
fi

# 如果文件存在, 暂时打印提示 (后续再加统计逻辑)
echo "File $CSV_PATH exists, ready for analysis"
```

图 2.15: 仅供第一问使用

3. 给脚本增加可执行权限

```
(base) xy_ai@Lenovo:~/q02$ chmod +x analyze.sh
```

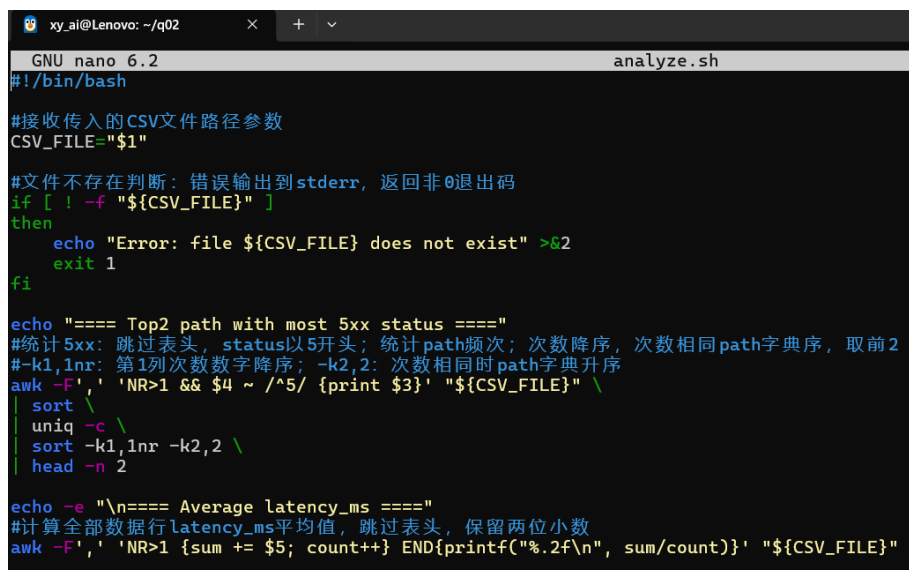
图 2.16: 否则报错 Permission denied

4. 文件不存在时将错误写入 stderr 并返回非零退出码

```
(base) xy_ai@Lenovo:~/q02$ ./analyze.sh no_exist.csv
Error: file no_exist.csv does not exist
(base) xy_ai@Lenovo:~/q02$ echo $?
1
(base) xy_ai@Lenovo:~/q02$ ./analyze.sh access.csv
File access.csv exists, ready for analysis
(base) xy_ai@Lenovo:~/q02$ echo $?
0
```

图 2.17

5. 使用 Shell 管道以及 awk、sort、head 等工具, 输出 HTTP 5xx 数量最多的前 2 个 path; 按次数降序, 次数相同时按 path 字典序



```
xy_ai@Lenovo: ~/q02
GNU nano 6.2 analyze.sh
#!/bin/bash

#接收传入的CSV文件路径参数
CSV_FILE="$1"

#文件不存在判断：错误输出到stderr，返回非0退出码
if [ ! -f "${CSV_FILE}" ]
then
    echo "Error: file ${CSV_FILE} does not exist" >&2
    exit 1
fi

echo "==== Top2 path with most 5xx status ====="
#统计5xx: 跳过表头, status以5开头; 统计path频次; 次数降序, 次数相同path字典序, 取前2
#-k1,1nr: 第1列次数数字降序; -k2,2: 次数相同时path字典升序
awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}' "${CSV_FILE}" \
| sort \
| uniq -c \
| sort -k1,1nr -k2,2 \
| head -n 2

echo -e "\n==== Average latency_ms ====="
#计算全部数据行latency_ms平均值, 跳过表头, 保留两位小数
awk -F',' 'NR>1 {sum += $5; count++} END{printf("%.2f\n", sum/count)}' "${CSV_FILE}"
```

图 2.18

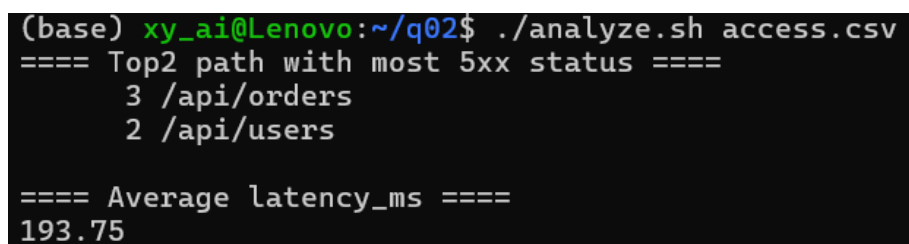
6. 给脚本增加可执行权限



```
(base) xy_ai@Lenovo:~/q02$ chmod +x analyze.sh
```

图 2.19: 否则报错 Permission denied

7. 输出全部数据行的平均 latency_ms, 保留两位小数; 表头不得计入。



```
(base) xy_ai@Lenovo:~/q02$ ./analyze.sh access.csv
==== Top2 path with most 5xx status ====
      3 /api/orders
      2 /api/users

==== Average latency_ms ====
193.75
```

图 2.20

8. 分别对 access.csv 和一个不存在的文件运行脚本; 不得使用 Python、电子表格或手工统计

```
(base) xy_ai@Lenovo:~/q02$ ./analyze.sh access.csv
==== Top2 path with most 5xx status ====
      3 /api/orders
      2 /api/users

==== Average latency_ms ====
193.75
(base) xy_ai@Lenovo:~/q02$ ./analyze.sh not_exist.csv
Error: file not_exist.csv does not exist
(base) xy_ai@Lenovo:~/q02$ echo $?
1
```

图 2.21

2.3 第 3 题制造、解决并解释一次合并冲突

第 3 题 制造、解决并解释一次合并冲突

主题: Version Control and Git 建议用时: 12—15 分钟

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突。

题目要求

- 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。
- 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交。
- 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed。
- 确认工作区干净，运行 git log --all --graph --decorate --oneline 查看历史。

图 2.22: 题目信息

1. 创建 q03 并进入初始化仓库，将分支改名为 main

```
xy_ai@Lenovo: ~/q03
(base) xy_ai@Lenovo:~$ mkdir q03
(base) xy_ai@Lenovo:~$ cd q03
(base) xy_ai@Lenovo:~/q03$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:     git config --global init.defaultBranch <name>
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:     git branch -m <name>
Initialized empty Git repository in /home/xy_ai/q03/.git/
(base) xy_ai@Lenovo:~/q03$ git branch -m master main
(base) xy_ai@Lenovo:~/q03$
```

图 2.23

2. main 分支初始提交，创建 config.txt，内容 mode=normal

```
(base) xy_ai@Lenovo:~/q03$ echo "mode=normal" > config.txt
(base) xy_ai@Lenovo:~/q03$ git add config.txt
(base) xy_ai@Lenovo:~/q03$ git commit -m "init: mode=normal"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <xy_ai@Lenovo.localdomain>) not allowed
(base) xy_ai@Lenovo:~/q03$ git config user.name "xy_ai"
(base) xy_ai@Lenovo:~/q03$ git config user.email "xy_ai@example.com"
(base) xy_ai@Lenovo:~/q03$ git commit -m "init: mode=normal"
[main (root-commit) 3a0187b] init: mode=normal
1 file changed, 1 insertion(+)
create mode 100644 config.txt
```

图 2.24: 第一次没有输入名字和邮箱，后改进

3. 创建 feature-a 分支，修改为 mode=safe 提交

```
(base) xy_ai@Lenovo:~/q03$ git checkout -b feature-a
Switched to a new branch 'feature-a'
(base) xy_ai@Lenovo:~/q03$ echo "mode=safe" > config.txt
(base) xy_ai@Lenovo:~/q03$ git add config.txt
(base) xy_ai@Lenovo:~/q03$ git commit -m "feature-a: mode=safe"
[feature-a f3916ec] feature-a: mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)
(base) xy_ai@Lenovo:~/q03$
```

图 2.25

4. 切回初始 main，从原始 main 创建 feature-b 分支，修改 mode=fast 提交

```
(base) xy_ai@Lenovo:~/q03$ git checkout main
Switched to branch 'main'
(base) xy_ai@Lenovo:~/q03$ git checkout -b feature-b
Switched to a new branch 'feature-b'
(base) xy_ai@Lenovo:~/q03$ echo "mode=fast" > config.txt
(base) xy_ai@Lenovo:~/q03$ git add config.txt
(base) xy_ai@Lenovo:~/q03$ git commit -m "feature-b: mode=fast"
[feature-b ca23db2] feature-b: mode=fast
1 file changed, 1 insertion(+), 1 deletion(-)
(base) xy_ai@Lenovo:~/q03$
```

图 2.26

5. 切回 main，先合并 feature-a，这一步没有冲突，main 现在 config.txt 内容是 mode=safe

```
(base) xy_ai@Lenovo:~/q03$ git checkout main
Switched to branch 'main'
(base) xy_ai@Lenovo:~/q03$ git merge feature-a
Updating 3a0187b..f3916ec
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
(base) xy_ai@Lenovo:~/q03$
```

图 2.27

6. 继续合并 feature-b，触发合并冲突

```
(base) xy_ai@Lenovo:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
(base) xy_ai@Lenovo:~/q03$
```

图 2.28

7. 解决冲突，保留 mode=safe，额外增加一行 note=reviewed

```
(base) xy_ai@Lenovo:~/q03$ nano config.txt
(base) xy_ai@Lenovo:~/q03$ git add config.txt
(base) xy_ai@Lenovo:~/q03$ git commit -m "解决冲突"
[main 4828a1a] 解决冲突
(base) xy_ai@Lenovo:~/q03$
```

图 2.29: 冲突解决，提交合并结果

```
GNU nano 6.2
<<<<<< HEAD
mode=safe
=====
mode=fast
>>>>>> feature-b
```

图 2.30: 按照题意改变 txt 之前

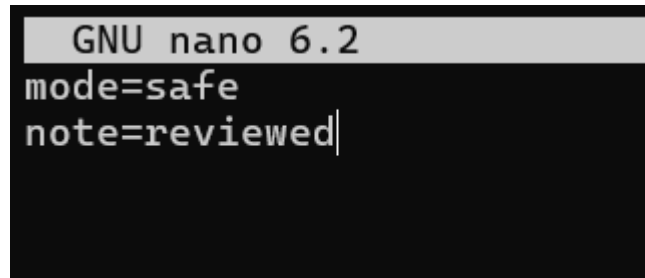


图 2.31: 按照题意改变 txt 之后

8. 确认工作区干净，按要求运行代码查看图形化提交日志

```
(base) xy_ai@Lenovo:~/q03$ git status
On branch main
nothing to commit, working tree clean
(base) xy_ai@Lenovo:~/q03$ git log --all --graph --decorate --oneline
* 4828a1a (HEAD -> main) 解决冲突
| \
| * ca23db2 (feature-b) feature-b: mode=fast
* | f3916ec (feature-a) feature-a: mode=safe
| /
* 3a0187b init: mode=normal
(base) xy_ai@Lenovo:~/q03$
```

图 2.32

2.4 第 4 题修复并构建一页技术说明

第 4 题 修复并构建一页技术说明

主题：LaTeX 文档编辑 建议用时：12—15 分钟

将下方模板保存为 q04/report.tex，补全内容并从命令行构建 PDF。

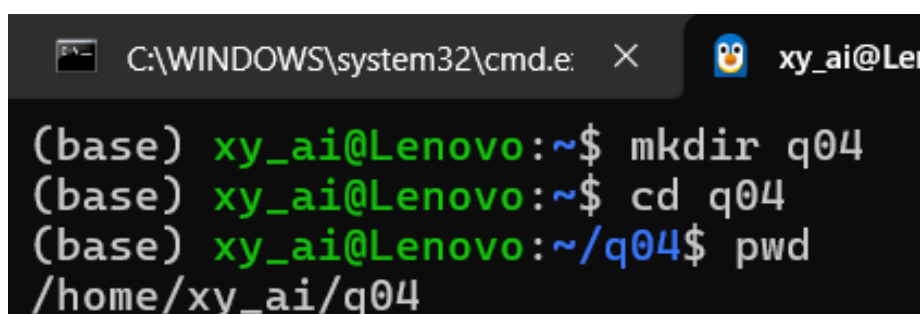
给定内容

```
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{姓名-学号}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\end{document}
```

题目要求

- 加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref 引用。
- 加入一个 2 行 2 列表格，设置 caption 和 label，并在正文中引用该表。
- 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf。
- 确认 PDF 能打开，且交叉引用不显示 “??” 。

1. 创建 q04 文件夹，进入目录



```
C:\WINDOWS\system32\cmd.e:  ×  xy_ai@Lenovo
(base) xy_ai@Lenovo:~$ mkdir q04
(base) xy_ai@Lenovo:~$ cd q04
(base) xy_ai@Lenovo:~/q04$ pwd
/home/xy_ai/q04
```

图 2.33

2. 在 wsl2 中下载 LaTeX

```
(base) xy_ai@lenovo:~/q04$ sudo apt update && sudo apt install -y texlive-xetex texlive-lang-chinese texlive-latex-extra latexmk
[sudo] password for xy_ai:
Get:1 file:/var/cuda-repo-wsl-ubuntu-13-3-local InRelease [1572 B]
Get:1 file:/var/cuda-repo-wsl-ubuntu-13-3-local InRelease [1572 B]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease
```

图 2.34

```
(base) xy_ai@Lenovo:~/q04$ latexmk --version
Latexmk, John Collins, 20 November 2021. Version 4.76
```

图 2.35: 验证下载成功

3. 创建符合要求的 report.tex

```
(base) xy_ai@Lenovo:~/q04$ nano report.tex
```

图 2.36

开始编写符合要求的 tex 内容，包含一个带编号和 label 的公式 $E = mc^2$ ，还有一个 2 行 2 列表格，设置 caption 和 label，均在正文按要求引用。

```
GNU nano 6.2 report.tex
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{Xiao Yang - 25020007140}
\maketitle

\section{Result}

The experimental formula is shown in Equation~\eqref{eq:sum}:
\begin{equation}
E = mc^2
\label{eq:sum}
\end{equation}

Experimental data are listed in Table~\ref{tab:data}:
\begin{table}[h]
\centering
\caption{Qingdao GDP Table}
\label{tab:data}
\begin{tabular}{|c|c|}
\hline
Year & GDP of Qingdao \\
\hline
2024 & 16719.46 \\
\hline
2025 & 17560.67 \\
\hline
\end{tabular}
\end{table}

Calculation is completed based on Equation~\eqref{eq:sum} and Table~\ref{tab:data}.
\end{document}
```

图 2.37

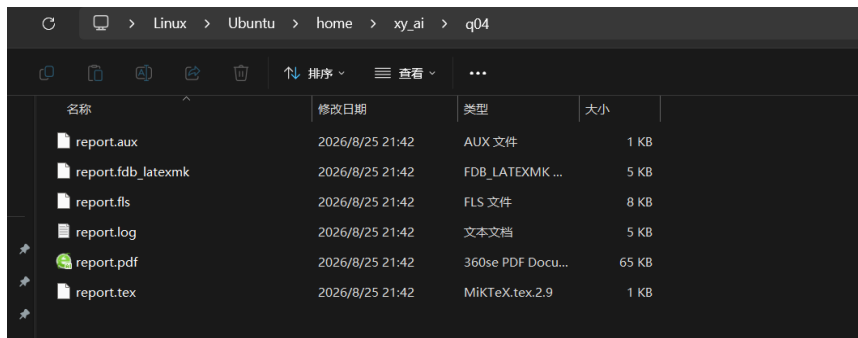
```
Calculation is completed based on Equation~\eqref{eq:sum} and Table~\ref{tab:data}.
```

图 2.38: 满足在正文引用的要求

4. 使用 `latexmk -pdf -halt-on-error` 生成 `report.pdf`

```
(base) xy_ai@Lenovo:~/q04$ latexmk -pdf -halt-on-error report.tex
Rc files read:
  /etc/LatexMk
  /home/xy_ai/.latexmkrc
Latexmk: This is Latexmk, John Collins, 20 November 2021, version: 4.76.
Latexmk: applying rule 'pdflatex'...
Rule 'pdflatex': File changes, etc:
  Changed files, or newly in use since previous run(s):
    'report.tex'
-----
Run number 1 of rule 'pdflatex'
-----
Running 'pdflatex -halt-on-error -recorder "report.tex"'
-----
```

图 2.39



名称	修改日期	类型	大小
report.aux	2026/8/25 21:42	AUX 文件	1 KB
report.fdb_latexmk	2026/8/25 21:42	FDB_LATEXMK ...	5 KB
report.fls	2026/8/25 21:42	FLS 文件	8 KB
report.log	2026/8/25 21:42	文本文档	5 KB
report.pdf	2026/8/25 21:42	360se PDF Docu...	65 KB
report.tex	2026/8/25 21:42	MiKTeX.tex.2.9	1 KB

图 2.40: 确认生成 pdf

5. 确认 PDF 能打开，且交叉引用不显示 “??”

Tool Report

Xiao Yang - 25020007140

August 25, 2026

1 Result

The experimental formula is shown in Equation (1):

$$E = mc^2 \tag{1}$$

Experimental data are listed in Table 1:

Table 1: Qingdao GDP Table

Year	GDP of Qingdao
2024	16719.46
2025	17560.67

Calculation is completed based on Equation (1) and Table 1.

图 2.41

Chapter 3

课后练习部分

3.1 课后第 1 题

`ls` 的 `-l` 选项 (flag) 作用是什么? 运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么? (提示: `man ls`)

1. 运行要求代码并观察输出

```
(base) xy_ai@Lenovo:~$ ls -l /
total 2868
drwxr-xr-x  3 root root    4096 Aug 20 20:37 Docker
lrwxrwxrwx  1 root root      7 Oct 26 2022 bin -> usr/bin
drwxr-xr-x  2 root root    4096 Apr 18 2022 boot
drwxr-xr-x  8 root root   3200 Aug 30 15:37 dev
drwxr-xr-x 86 root root    4096 Aug 30 16:12 etc
drwxr-xr-x  3 root root    4096 Jul 26 17:41 home
-rwxr-xr-x  1 root root 2836528 Jul 24 08:46 init
lrwxrwxrwx  1 root root      7 Oct 26 2022 lib -> usr/lib
lrwxrwxrwx  1 root root      9 Oct 26 2022 lib32 -> usr/lib32
lrwxrwxrwx  1 root root      9 Oct 26 2022 lib64 -> usr/lib64
lrwxrwxrwx  1 root root     10 Oct 26 2022 libx32 -> usr/libx32
drwx----- 2 root root   16384 Jul 26 23:03 lost+found
drwxr-xr-x  2 root root    4096 Oct 26 2022 media
drwxr-xr-x  7 root root    4096 Aug 18 21:29 mnt
drwxr-xr-x  3 root root    4096 Aug 20 11:07 opt
dr-xr-xr-x 312 root root      0 Aug 30 15:26 proc
drwx----- 4 root root    4096 Aug 20 20:37 root
drwxr-xr-x  7 root root    140 Aug 30 15:26 run
lrwxrwxrwx  1 root root      8 Oct 26 2022 sbin -> usr/sbin
drwxr-xr-x  8 root root    4096 Oct 26 2022 snap
drwxr-xr-x  2 root root    4096 Oct 26 2022 srv
dr-xr-xr-x 13 root root      0 Aug 30 15:26 sys
drwxrwxrwt 64 root root   12288 Aug 30 16:13 tmp
drwxr-xr-x 14 root root    4096 Oct 26 2022 usr
drwxr-xr-x 14 root root    4096 Aug 20 10:59 var
drwx----- 2 root root    4096 Aug 27 14:47 wslIgJikD
drwx----- 2 root root    4096 Aug 27 14:47 wslbfMIaC
drwx----- 2 root root    4096 Aug 27 14:47 wslfOBhiD
drwx----- 2 root root    4096 Aug 27 14:47 wslgOEekD
drwx----- 2 root root    4096 Aug 27 14:47 wslokoMkD
(base) xy_ai@Lenovo:~$
```

图 3.1

2. 回答问题:

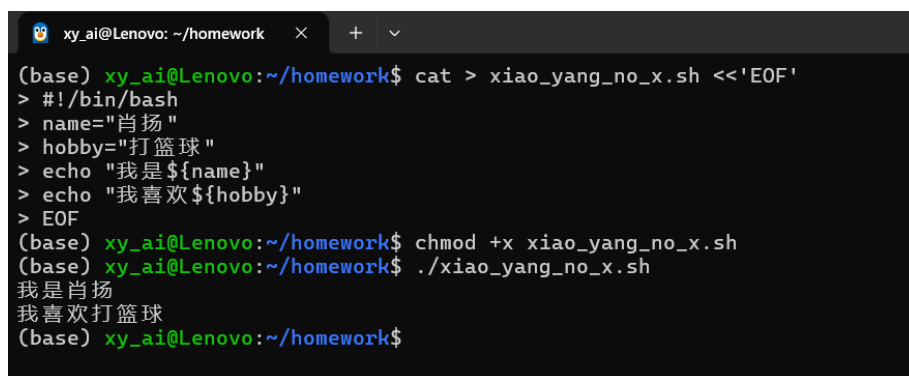
ls 命令的-l 选项用于以长列表格式展示文件的详细信息。输出结果每行开头的 10 个字符用来描述文件类型与权限，其中第一位代表文件类型，-代表普通文件，d 代表目录，l 代表软链接，b 代表块设备，c 代表字符设备；第 2 至 4 位是文件属主也就是所有者的权限，分别对应读、写、执行三种权限；第 5 至 7 位是文件所属用户组的权限；第 8 至 10 位代表系统中其他所有用户的权限，r 代表可读，w 代表可写，x 代表可执行，通过这三段权限位，就定义了三类不同身份用户对该文件拥有的访问能力。

3.2 课后第 2 题

问题如下：

在脚本的 set 选项（flag）里加入 -x 会发生什么？写个简单脚本试试并观察输出。

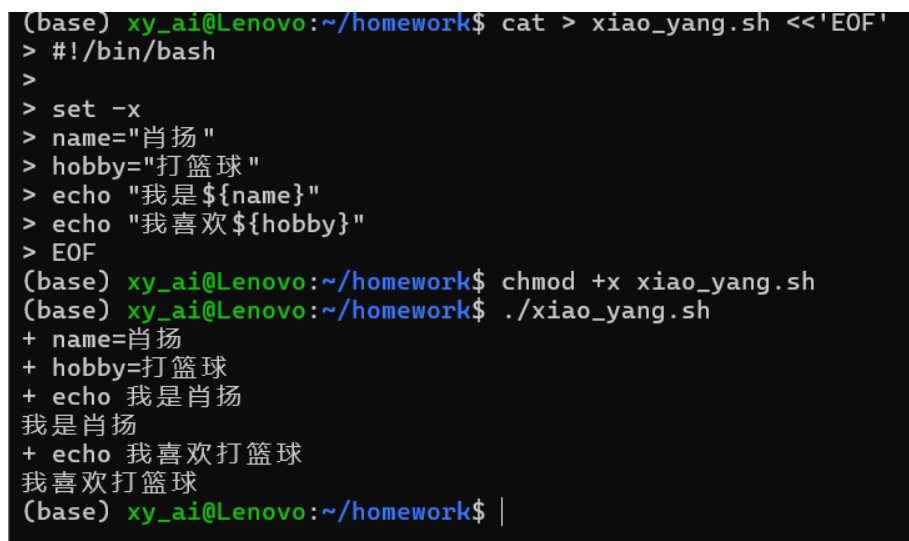
1. 对比：不加入 -x 版本



```
(base) xy_ai@Lenovo: ~/homework $ cat > xiao_yang_no_x.sh <<'EOF'
> #!/bin/bash
> name="肖扬"
> hobby="打篮球"
> echo "我是${name}"
> echo "我喜欢${hobby}"
> EOF
(base) xy_ai@Lenovo:~/homework$ chmod +x xiao_yang_no_x.sh
(base) xy_ai@Lenovo:~/homework$ ./xiao_yang_no_x.sh
我是肖扬
我喜欢打篮球
(base) xy_ai@Lenovo:~/homework$
```

图 3.2

2. 对比：加入 -x 版本



```
(base) xy_ai@Lenovo:~/homework$ cat > xiao_yang.sh <<'EOF'
> #!/bin/bash
>
> set -x
> name="肖扬"
> hobby="打篮球"
> echo "我是${name}"
> echo "我喜欢${hobby}"
> EOF
(base) xy_ai@Lenovo:~/homework$ chmod +x xiao_yang.sh
(base) xy_ai@Lenovo:~/homework$ ./xiao_yang.sh
+ name=肖扬
+ hobby=打篮球
+ echo 我是肖扬
我是肖扬
+ echo 我喜欢打篮球
我喜欢打篮球
(base) xy_ai@Lenovo:~/homework$ |
```

图 3.3

3. 学到的内容：

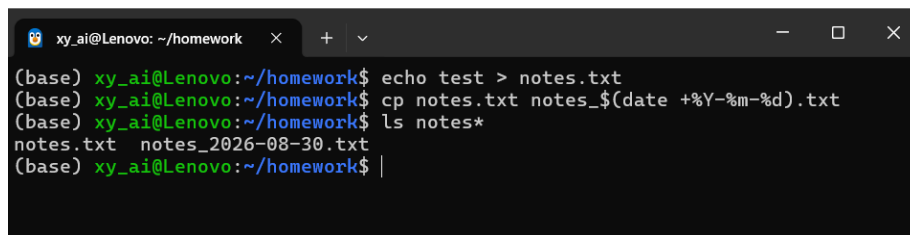
set -x 是 bash 脚本的调试选项，开启后 shell 会在执行每条命令前，输出经过变量、参数展开后的实际命令，输出行以加号 + 作为标记，方便我们观察脚本真实执行的内容；没有 set -x 时脚本只会输出 echo 等命令打印的结果，看不到变量赋值这类内部执行流程

3.3 课后第 3 题

问题如下：

写一条命令，把文件复制为带当天日期的备份文件名，例如 notes.txt 变为 notes_2026-01-12.txt

1. 先创建测试文件, 再复制，查看文件

A terminal window titled 'xy_ai@Lenovo: ~/homework' with standard window controls. It shows a sequence of commands and their outputs: 1. 'echo test > notes.txt' creates a file. 2. 'cp notes.txt notes_\$(date +%Y-%m-%d).txt' creates a backup with a date-based filename. 3. 'ls notes*' lists the files, showing 'notes.txt' and 'notes_2026-08-30.txt'.

```
(base) xy_ai@Lenovo:~/homework$ echo test > notes.txt
(base) xy_ai@Lenovo:~/homework$ cp notes.txt notes_$(date +%Y-%m-%d).txt
(base) xy_ai@Lenovo:~/homework$ ls notes*
notes.txt  notes_2026-08-30.txt
(base) xy_ai@Lenovo:~/homework$
```

图 3.4

3.4 课后第 4 题

题目如下：

写一个脚本，接收文件名参数 (\$1)，用 test -f 或 [-f ...] 检查该文件是否存在，并根据结果输出不同提示。

1. 创建脚本：nano checkfile.sh

```
xy_ai@Lenovo: ~/homework
GNU nano 6.2
#!/bin/bash

# $1 获取传入的第一个参数
filename="$1"

# 判断是否为普通文件
if [ -f "$filename" ]; then
    echo "文件$filename 存在。"
else
    echo "文件$filename 不存在。"
fi
```

图 3.5

2. 添加权限，测试存在和不存在的文件，检查输出结果

```
(base) xy_ai@Lenovo:~$ cd homework
(base) xy_ai@Lenovo:~/homework$ nano checkfile.sh
(base) xy_ai@Lenovo:~/homework$ nano checkfile.sh
(base) xy_ai@Lenovo:~/homework$ chmod +x checkfile.sh
(base) xy_ai@Lenovo:~/homework$ ./checkfile.sh notes.txt
文件notes.txt 存在。
(base) xy_ai@Lenovo:~/homework$ ./checkfile.sh abc.txt
文件abc.txt 不存在。
(base) xy_ai@Lenovo:~/homework$ |
```

图 3.6

3.5 课后第 5 题

使用管道找出你「home 目录」中最常见的 5 种文件扩展名。(提示：组合 find 、 grep / sed / awk、sort、uniq -c 以及 head)

1. 如下：

```
(base) xy_ai@Lenovo:~$ find ~ -type f 2>/dev/null | sed -E 's/.*\.[^.]+\$/\1/' | sort | uniq -c | sort -nr | head -n5
40251 py
30656 pyc
20320 h
15980 pyi
6677 js
(base) xy_ai@Lenovo:~$ |
```

图 3.7

3.6 课后第 6 题

问题如下：克隆这门课程网站对应的代码仓库
包含三个小问题：

1. 将版本历史以图形的形式可视化，浏览版本提交历史。
2. 最后一个修改 README.md 文件的人是谁？（提示：给 git log 加上参数）
3. _config.yml 文件中 collections: 这一行，最后一次修改对应的提交信息是什么？（提示：使用 git blame 和 git show）

1. 克隆库后进入

```
(base) xy_ai@Lenovo:~/homework$  
(base) xy_ai@Lenovo:~/homework$ echo test > notes.txt  
(base) xy_ai@Lenovo:~/homework$ cp notes.txt notes_$(date +%Y-%m-%d).txt  
(base) xy_ai@Lenovo:~/homework$ ls notes*  
notes.txt  notes_2026-08-30.txt  
(base) xy_ai@Lenovo:~/homework$  
(base) xy_ai@Lenovo:~/homework$ git clone https://github.com/missing-semester/missing-semester.git  
Cloning into 'missing-semester'...  
remote: Enumerating objects: 3455, done.  
remote: Counting objects: 100% (224/224), done.  
remote: Compressing objects: 100% (134/134), done.  
remote: Total 3455 (delta 130), reused 95 (delta 90), pack-reused 3231 (from 4)  
Receiving objects: 100% (3455/3455), 12.81 MiB | 8.09 MiB/s, done.  
Resolving deltas: 100% (2084/2084), done.  
(base) xy_ai@Lenovo:~/homework$ cd missing-semester
```

图 3.8

2. 将版本历史可视化图形展示

```
xy_ai@Lenovo: ~/homework/r
(base) xy_ai@Lenovo:~/homework/missing-semester$ git log --all --graph --decorate --oneline
* fea9192 (HEAD -> master, origin/master, origin/HEAD) Merge branch 'oiahoon/docs/fix-uv-lock-example'
/* 7343431 docs: fix uv lockfile Docker example
* 853e0ea Merge branch 'tufailrizvi-debug/fixing_typo'
/* 2df4a16 Fix typos in SSH-related documentation
* adaebe3 Merge branch 'ariuxklc/master'
/* de7ac78 Add Mongolian translation link
* 575bf9f Bump actions/checkout from 6 to 7 in the actions group (#385)
* f71203e Merge pull request #382 from JasonLuo-4/fix-lecture2-typos
/* 8b51b99 Fix two typos in Lecture 2 exercises
* def3237 Merge branch 'arteen1000/patch-1'
/* 495a201 Change wording slightly in code quality lecture
/* 49f676c Tweak text about license
* 5a501b8 Merge branch 'itiquette/fix/url-change'
/* 4f14f71 Switch to new url for swedish
* 46e3a32 Fix broken link
/* 600ade0 Merge branch 'huntagain2008/huntagain2008-patc...'
/* 376db0b Fix the greet CLI tool
* c714fda Merge branch 'dylantran035/fix-shell-typo'
/* 57626a3 Fix confusing phrasing in shell.md
* 3822e28 Merge branch 'huntagain2008/huntagain2008-patc...'
/* 19bcac2 Switch to 'docker compose'
* 6df931a Merge branch 'xiqi98/fix-typo'
/* 2c86ace Fix typo: remove duplicate "the" in beyond-code.md
```

图 3.9

3. 查询最后修改 README.md 的人

```
(base) xy_ai@Lenovo:~/homework/missing-semester$ git log -1 --pretty=format:"%an" README.md
Anish Athalye
```

图 3.10

4. git blame, 定位这一行的 commit 哈希, 然后 git show, 查看该提交完整信息

```
xy_ai@Lenovo: ~/homework/r x + v
(base) xy_ai@Lenovo:~/homework/missing-semester$ git blame _config.yml | grep "collections:"
a88b4eac (Anish Athalye 2020-01-17 15:26:30 -0500 19) collections:
(base) xy_ai@Lenovo:~/homework/missing-semester$ git show a88b4eac
commit a88b4eac326483e29bdac5ee0a39b180948ae7fc
Author: Anish Athalye <me@anishathalye.com>
Date: Fri Jan 17 15:26:30 2020 -0500

    Redo lectures as a collection

diff --git a/2020/index.html b/2020/index.html
deleted file mode 100644
index 153ddc8..0000000
--- a/2020/index.html
+++ /dev/null
@@ -1,5 +0,0 @@
-----
- layout: redirect
- redirect: /
- title: Missing Semester 2020
-----
diff --git a/2019/automation.md b/_2019/automation.md
similarity index 100%
rename from 2019/automation.md
rename to _2019/automation.md
diff --git a/2019/backups.md b/_2019/backups.md
similarity index 100%
rename from 2019/backups.md
rename to _2019/backups.md
diff --git a/2019/command-line.md b/_2019/command-line.md
similarity index 100%
rename from 2019/command-line.md
rename to _2019/command-line.md
diff --git a/2019/course-overview.md b/_2019/course-overview.md
similarity index 100%
rename from 2019/course-overview.md
rename to _2019/course-overview.md
diff --git a/2019/data-wrangling.md b/_2019/data-wrangling.md
similarity index 100%
rename from 2019/data-wrangling.md
rename to _2019/data-wrangling.md
diff --git a/2019/dotfiles.md b/_2019/dotfiles.md
similarity index 100%
rename from 2019/dotfiles.md
rename to _2019/dotfiles.md
diff --git a/2019/editors.md b/_2019/editors.md
similarity index 100%
rename from 2019/editors.md
rename to _2019/editors.md
(base) xy_ai@Lenovo:~/homework/missing-semester$ |
```

图 3.11

3.7 课后第 7 题

题目 1：文档基础结构（标题、作者）

考点：文档框架、`\title`、`\author`、`\maketitle`

要求：编写一份 LaTeX 文档

1. 文档类型为 `article`
2. 设置标题：Git 版本控制入门
3. 设置作者：肖扬
4. 正文写一句话：Git 是一款强大的版本管理工具。
5. 生成标题并完成文档。

1. LaTeX 代码和展示 PDF 如下：



图 3.12

3.8 课后第 8 题

题目 2：文本格式与分段（加粗、换行分段）

考点：段落、`\textbf{}` 加粗，不需要设置标题

要求：编写一份 LaTeX 文档

- 1. 文档类型 `article`
- 2. 第一段：Shell 脚本可以用来自动化执行任务。
- 3. 第二段：使用 `\textbf{set -x}` 可以开启 Shell 脚本调试，打印展开后的执行命令。
- 4. 两个内容分为两个独立段落输出。

1. LaTeX 代码如下：

```
LaTeX 1.tex X LaTeX2.tex X
LaTeX2.tex
1 \documentclass{ctexart}
2 \begin{document}
3 Shell脚本可以用来自动化执行任务。
4
5 使用\textbf{set -x}可以开启Shell脚本调试，打印展开后的执行命令。
6 \end{document}
7
```

图 3.13

2. 呈现的 PDF 如下：

Shell 脚本可以用来自动化执行任务。
使用 `set -x` 可以开启 Shell 脚本调试，打印展开后的执行命令。

图 3.14

3.9 课后第 9 题

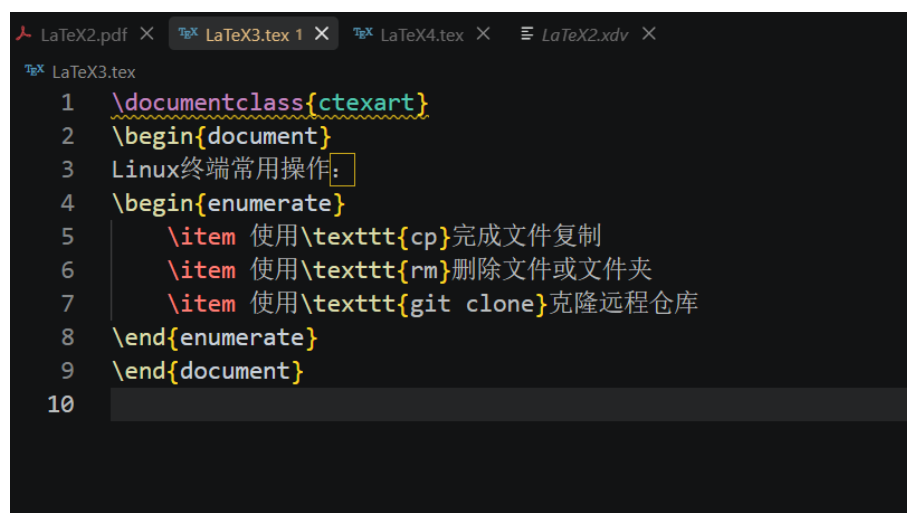
题目 3：有序列表环境 `enumerate`

考点：`enumerate` 有序列表环境，`\item`

要求：编写一份 LaTeX 文档

1. 文档类型 `article`
2. 正文输出文字：Linux 终端常用操作：
3. 使用有序列表列出下面 3 点
 1. 使用 `cp` 完成文件复制
 2. 使用 `rm` 删除文件或文件夹
 3. 使用 `git clone` 克隆远程仓库

1. LaTeX 代码如下：



```
LaTeX2.pdf X LaTeX3.tex 1 X LaTeX4.tex X LaTeX2.xdv X
LaTeX3.tex
1 \documentclass{ctexart}
2 \begin{document}
3 Linux终端常用操作:
4 \begin{enumerate}
5   \item 使用\texttt{cp}完成文件复制
6   \item 使用\texttt{rm}删除文件或文件夹
7   \item 使用\texttt{git clone}克隆远程仓库
8 \end{enumerate}
9 \end{document}
10
```

图 3.15

2. 呈现的 PDF 如下：

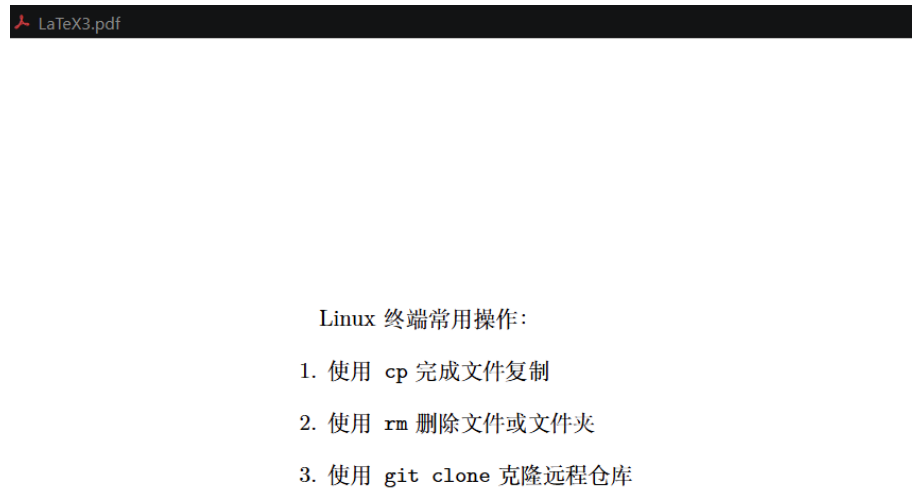


图 3.16

3.10 课后第 10 题

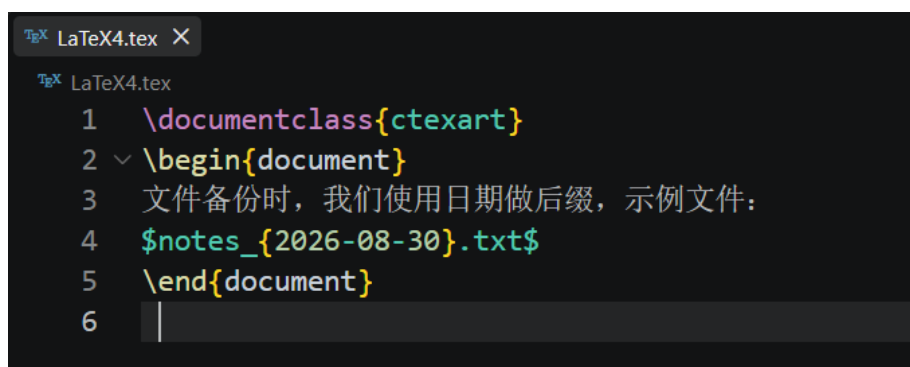
题目 4：行内数学公式、下标语法

考点：行内公式 $\$ \$$ ，下标 `_{}`

要求：编写一份 LaTeX 文档

1. 文档类型 `article`
2. 正文文字：文件备份时，我们使用日期做后缀，示例文件：
3. 使用行内公式写出： `notes_{2026-08-30}.txt`
4. 将示例文件名放在公式环境内。

1. LaTeX 代码如下：



```
TeX LaTeX4.tex X
TeX LaTeX4.tex
1 \documentclass{ctexart}
2 \begin{document}
3 文件备份时，我们使用日期做后缀，示例文件：
4 $notes_{2026-08-30}.txt$
5 \end{document}
6
```

图 3.17

2. 呈现的 PDF 如下：



图 3.18

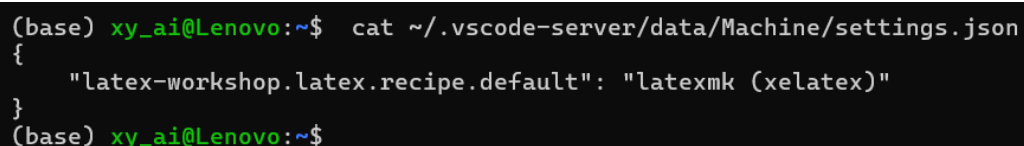
Chapter 4

解题感悟

4.1 课上练习

1. 解决问题 1: 使用 LaTeX 遇到无法输出汉字问题

在使用 LaTeX 编写时, 发现使用汉字会出现汉字无法输出的问题, 通过写入 "latex-workshop.latex.recipe.default": "latexmk (xelatex)" 解决问题



```
(base) xy_ai@Lenovo:~$ cat ~/.vscode-server/data/Machine/settings.json
{
  "latex-workshop.latex.recipe.default": "latexmk (xelatex)"
}
(base) xy_ai@Lenovo:~$
```

图 4.1: 相关图片

2. 解决问题 2: 使用 LaTeX 遇到英文符号自动转化为中文

发现使用 "texttt" 打字机字体环境, 引号不会被 ctex 转换, 可以原样输出英文引号。

3. 解决问题 3: 区分 ls -la input 和 ls -laR input

ls -la input: 只看 input 这一层, 不会进入 docs、tmp 子目录

ls -laR input: 递归往下钻, 把子文件夹内部所有文件全部列出来, 截图里就打印出 input/docs、input/tmp 下面的文件了, 包括隐藏文件.secret.txt。

4. 在文件整理任务中, 我学会了使用 find 查找指定类型的文件, 并结合 cpio 在复制文件时保留原有的目录结构。同时, 我进一步理解了目录权限和普通文件权限的区别, 以及 750、640 等数字权限所代表的实际含义。通过比较 ls -la input 和 ls -laR input, 我也认识到普通查看与递归查看之间的区别。
5. 在日志统计任务中, 我体会到了 Shell 管道的作用。将 awk、sort、head 等工具组合起来, 可以完成筛选、统计、排序和输出等一系列操作。相比手工统计, 这种方法效率更高, 也更适合处理大量数据。脚本对文件是否存在进行判断, 并通过标准错误和非零退出码报告异常, 也让我认识到一个完整的脚本不仅要能够得到正确结果, 还应当具备基本的错误处理能力。
6. Git 合并冲突练习让我理解了分支之间产生冲突的原因。当两个分支修改同一位置时, Git 无法自动判断应当保留哪一项内容, 需要开发者检查冲突标记、确定最终内容并重新提交。这次练习使我不再把合并冲突简单地看成报错, 而是把它理解为多人协作过程中需要人工作出选择的一种正常情况。
7. 在 LaTeX 练习中, 我掌握了公式、表格、标题和交叉引用

的基本写法，也认识到 LaTeX 文档通常需要经过完整编译才能正确生成引用结果。课堂内容涵盖了文件管理、数据处理、版本控制和文档构建等多个方面，使我认识到这些工具虽然用途不同，但都强调操作规范、过程可验证和结果可复现。

4.2 课后练习

1. 在完成文件检查脚本时，脚本文件已经创建并写入了正确内容，但直接运行时出现了“Permission denied”的提示。检查后发现，脚本只有读写权限，没有执行权限。使用 `chmod +x checkfile.sh` 为脚本增加执行权限后，再通过 `./checkfile.sh` 文件名运行，脚本便能够正常执行。这个问题让我认识到，脚本内容正确并不代表一定能够直接运行，还需要检查文件权限。
2. 在测试文件存在性检查时，如果文件名中带有空格，直接使用未加引号的 `1 Shell` [-f"1"]。修改后，即使文件名中包含空格，脚本也能把它作为一个完整路径处理。通过这个问题，我进一步理解了 Shell 脚本中为变量添加引号的重要性。
3. 在查询 `_config.yml` 中 `collections:` 这一行的最后修改记录时，最初只查看普通的 `git log`，得到的是整个仓库的提交历史，无法直接判断具体是哪一次提交修改了这一行。后来先使用 `git blame _config.yml` 定位该行对应的提交哈希，再使用 `git show` 提交哈希查看提交作者、时间、说明和修改内容，最终找到了需要的信息。这个问题让我明白，不同 Git 命令解决的问题不同：`git log` 用于查看历史，`git blame` 用于追踪具体代码行，`git show` 用于查看某次提交的详细内容。
4. 课后练习心得总结如下：课后练习进一步巩固了课堂知识，

并让我有机会独立完成更加具体的操作。通过分析 `ls -l` 输出，我能够根据开头的字符判断文件类型，并分别识别文件所有者、用户组和其他用户的读、写、执行权限。通过对比是否启用 `set -x`，我了解到它可以显示脚本中经过变量和参数展开后实际执行的命令，因此在排查脚本问题时非常有帮助。日期备份、文件存在性检查和常见扩展名统计等练习，使我更加熟悉变量、条件判断和管道组合的使用。尤其是在统计文件扩展名时，需要把多个功能单一的命令连接起来，才能完成查找、提取、排序和计数。这让我体会到 Linux 命令行的特点不是依赖某一个复杂命令，而是通过组合多个简单工具来解决问题。Git 课后练习让我学会了从不同角度查看仓库历史。`git log` 适合查看整体提交记录，`git log - README.md` 可以查找与指定文件有关的提交，而 `git blame` 可以定位某一行最后由哪个提交修改，再通过 `git show` 查看该提交的详细信息。后面的 LaTeX 练习则进一步加深了我对文档结构、公式、表格和排版效果之间关系的理解。总体来看，课后练习提高了我独立分析问题的能力。完成任务时不能只照着命令输入，还要观察运行结果、理解命令参数，并在出现异常时逐步定位原因。只有真正理解每一步的作用，才能在题目条件发生变化时灵活调整解决方法。

Chapter 5

上传 GitHub

5.1 编写.gitignore

在把课程作业上传到 GitHub 之前，我对文件夹做了清理，并在仓库根目录写入了如下 .gitignore：

LaTeX 编译中间产物

```
*.aux  
*.fdb_latexmk  
*.fls  
*.log  
*.synctex.gz  
*.xdv  
*.toc
```

Windows 下载标记

```
*Zone.Identifier*
```

个人 IDE 配置

```
.vscode/
```

逐条说明如下：

- *.aux: 交叉引用辅助文件, 每次编译自动生成;
- *.fdb_latexmk、*.fls: latexmk 记录的依赖关系和文件清单, 属于工具内部数据;
- *.log: 编译日志, 只用于排查错误;
- *.synctex.gz: TeX 源码与 PDF 之间双向定位的压缩文件;
- *.xdv: xelatex 生成的中间格式, 在产出 PDF 前需要先经过它;
- *.toc: 目录数据文件, 由 \tableofcontents 生成;
- *Zone.Identifier*: Windows 的”下载来源”标记。文件从 Windows 复制进 WSL 时, NTFS 备用数据流会被拆成独立的 xxx:Zone.Identifier 小文件 (清理时共发现 324 个), 写进规则后以后不会再混入仓库;
- .vscode/: VS Code 的个人工作区配置, 不属于作业交付内容。

5.2 GitHub 仓库链接

作业仓库地址 (公开, 默认分支 main):

https://github.com/xiaoyang1088/system_development_tools

说明: 仓库名为 system_development_tools, GitHub 用户名 xiaoyang1088。仓库内按题目组织为 homework/、q01/ 至 q04/、实验报告/ 等目录, 根目录含 .gitignore。

5.3 commit 提交记录

注: 第四次提交时将更新这一部分